

Design Pack

Component, sequence and deployment diagrams with planning/execution separation

Deliverable 3 of 6 · Case Study no. 39 · PortFlow

Name	Sakshi Shingole	Roll No.	150096724056
Cohort	Jensen Huang Cohort	Course	Software Engineering & Project Management
Case	PortFlow - Container Terminal Yard and Gate Operations		

1. Design Overview

PortFlow is split into two layers that communicate only through **versioned plan snapshots** and a **message broker**:

- **Planning engine** (the "thinking"): heavy optimisation of yard positions, crane sequences and slot capacity. It can be slow or temporarily down.
- **Execution layer** (the "doing"): lightweight and real time. It issues crane jobs, validates trucks at the gate and records moves. It keeps a local copy of the last plan and simple fallback rules, so the terminal keeps working if the planner fails.

Why separate them? Optimisation is unpredictable in run time and prone to failure, while the gate and cranes must never wait. Separating them also lets each be deployed, scaled and tested independently.

2. Component Diagram

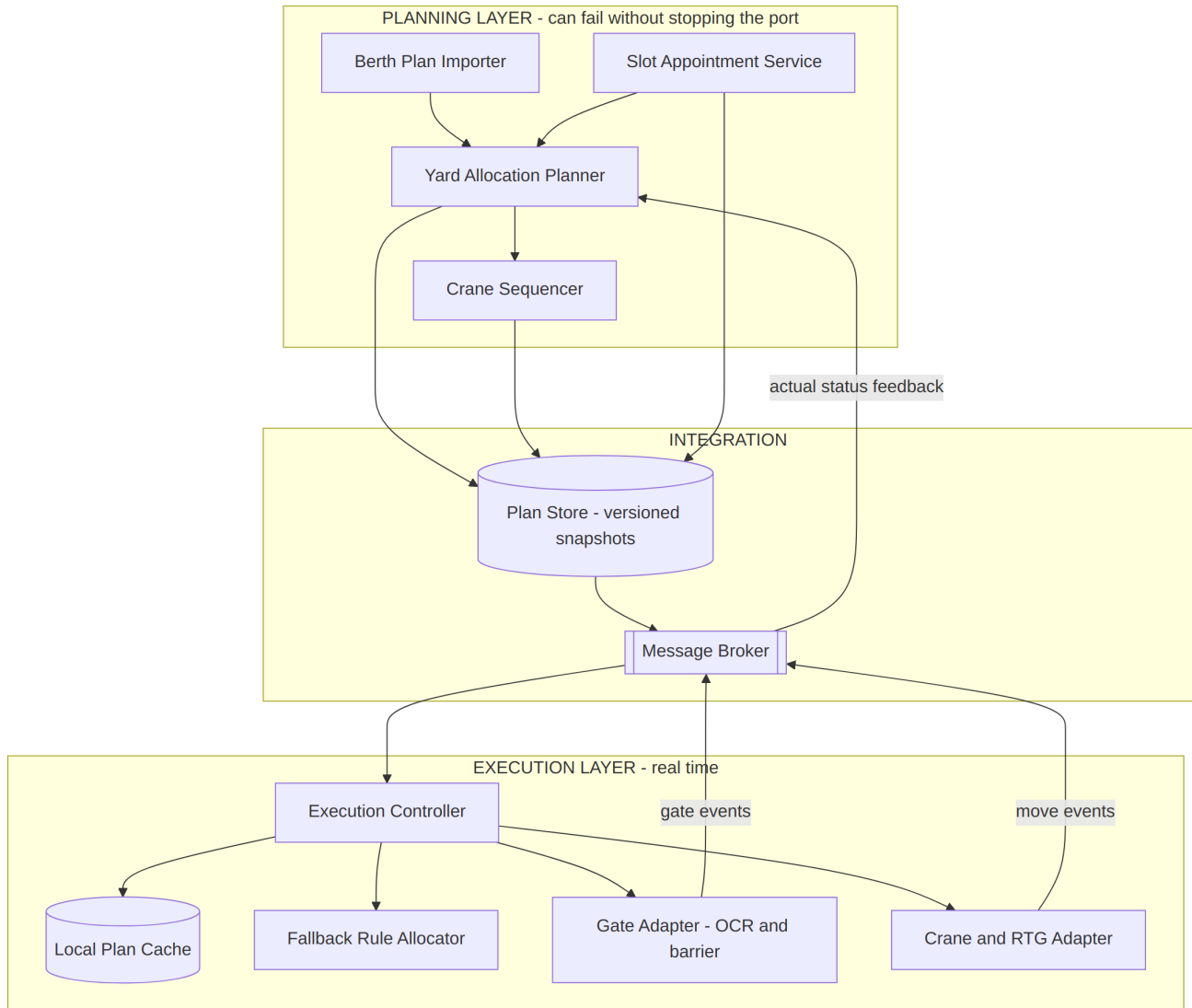


Figure 1: Component diagram. The planning layer publishes plans; the execution layer consumes them and feeds actual events back

Component responsibilities

Component	Responsibility
Berth Plan Importer	Brings the digital berth plan (vessel ETA/ETD, berth, cranes) into the planner
Yard Allocation Planner	Chooses a stack position for each container by departure order and type
Crane Sequencer	Orders crane jobs to minimise moves and re-handles
Slot Appointment Service	Books truck slots within gate, yard and crane capacity
Plan Store / Message Broker	Versioned plan snapshots and reliable event transport
Execution Controller	Issues jobs, validates trucks, records moves; uses cache and fallback if needed
Gate and Crane Adapters	Interface to OCR cameras, barriers, crane/RTG PLCs and tablets

3. Sequence Diagram: Truck Booking to Container Pick-up

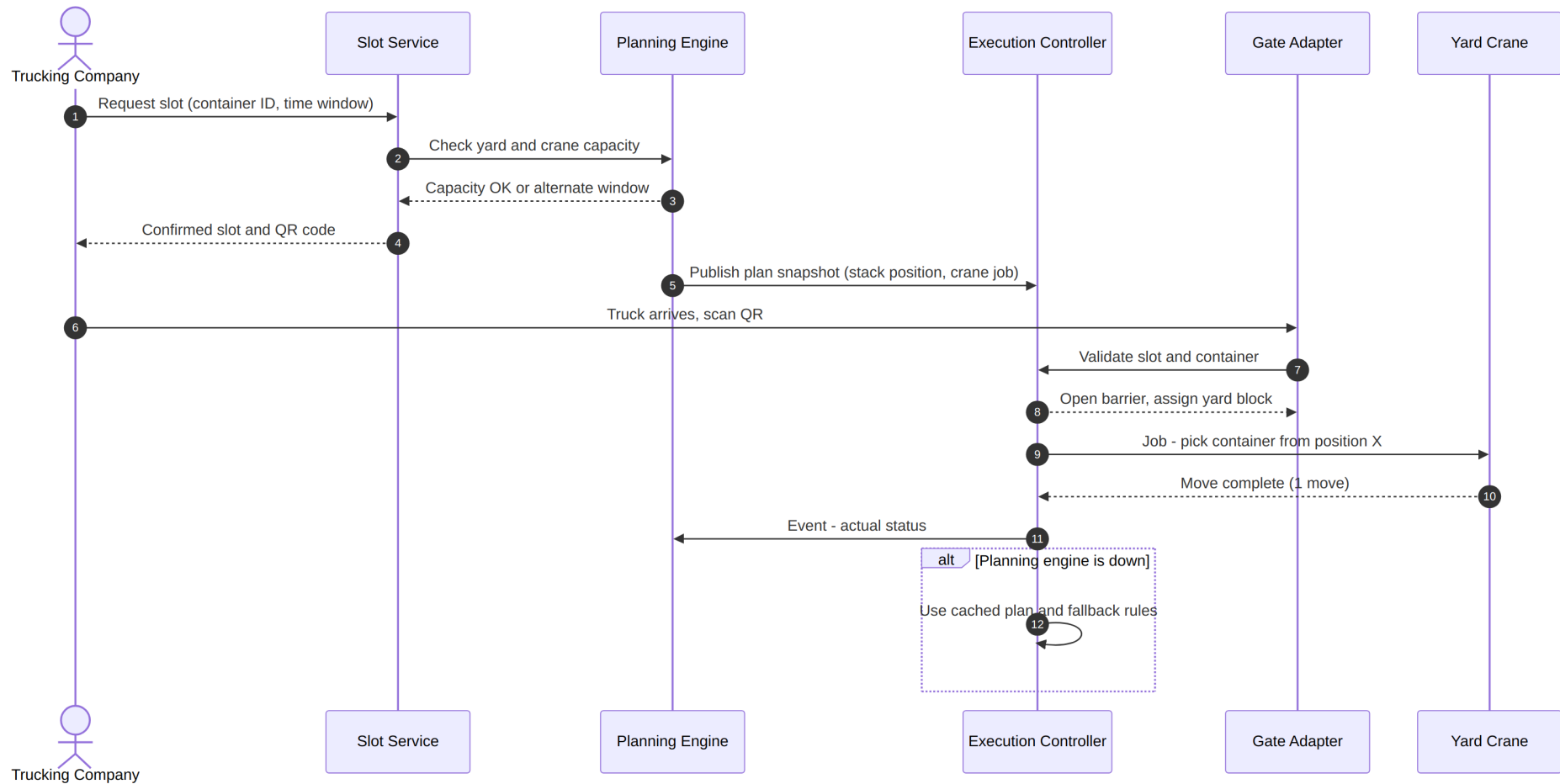


Figure 2: Sequence diagram. The alternative block shows execution continuing from the cached plan when the planning engine is down

4. Deployment Diagram

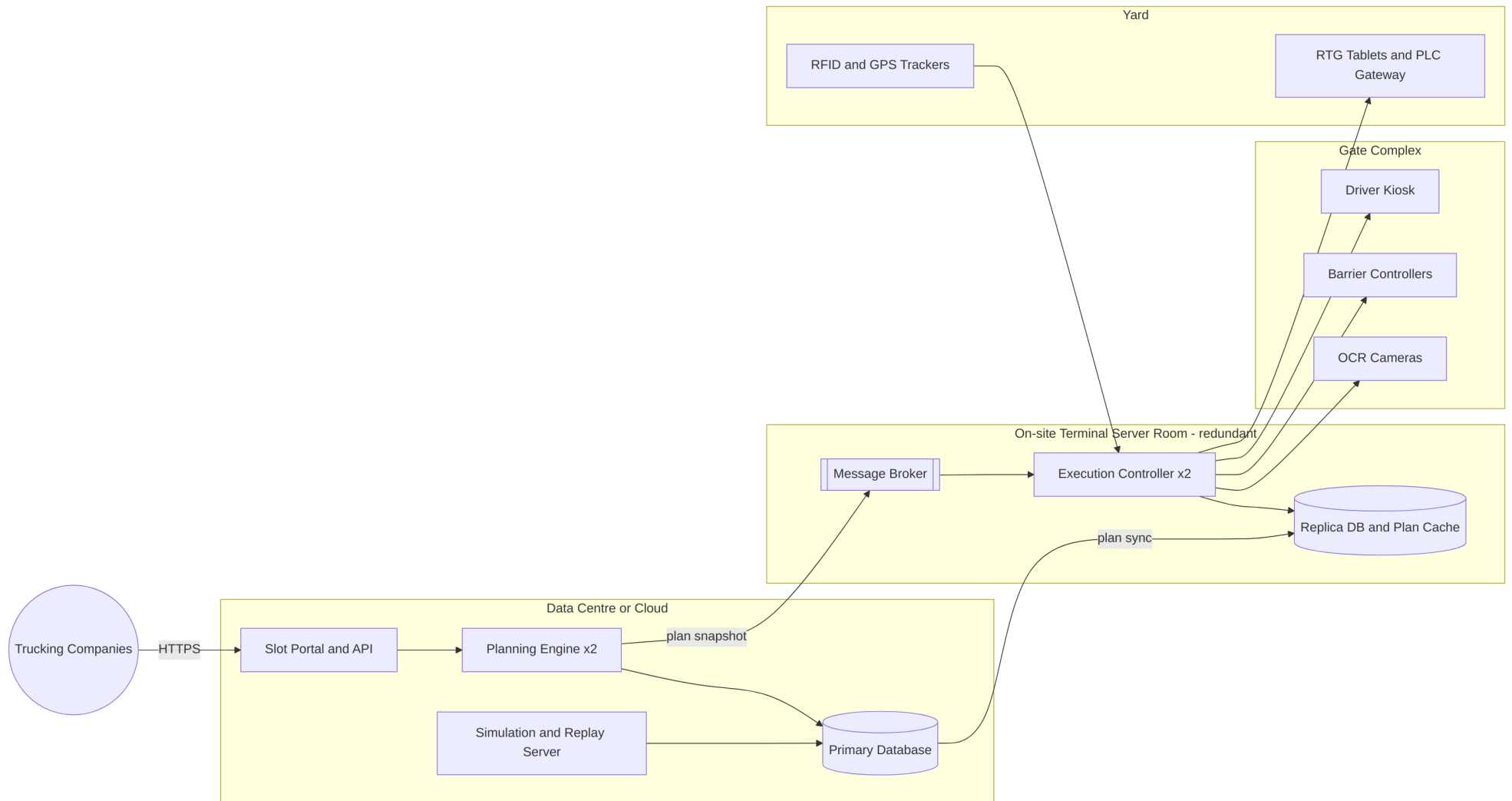


Figure 3: Deployment diagram. The on-site edge server room keeps the terminal running if the link to the data centre drops

5. Planning / Execution Separation: Failure Behaviour

Failure	What keeps working	What stops
Planning engine down	Gate, cranes and execution using last plan and fallback rules	New optimisation; new slot offers fall back to existing bookings
Link to data centre down	Everything on-site (replica DB and plan cache)	Online booking; carriers use phone or queue
One execution node down	Standby node takes over (x2)	Nothing (failover)

6. Key Design Decisions

Decision	Reason
Separate planning from execution	Planner failure must not stop the gate or cranes (EX-2, NFR-2)
Versioned plan snapshots	Execution always knows which plan it is following; supports audit (NFR-4)
Message broker between layers	Loose coupling and buffering during outages
Redundant on-site edge	Terminal runs even if the cloud link drops
Adapters for equipment	Isolates legacy crane/PLC interfaces, the highest integration risk (R1)